

Hw3

Packages

```
library(data.table)
library(ggplot2)
library(readr)
```

Problem 1

Part A. Solution

```
# helper functions
source("hw3.R")

## If you want correct answers, use rational arithmetic.
## See the Warnings sections in help pages for
##     functions that do computational geometry.

# main function
MC3search <- function(response, data, n_iter) {
  #'
  #' Args:
  #' ----
  #' - response (int): col index of binary response
  #' - data (data.frame | matrix): target dataset
  #' - n_iter (int): number of MC3 search iterations
  #'
  #' Returns:
  #' -----
  #' - list(best model, best AIC) (list)
  #'   - best model: set of index of explanatory vars
  #'   - best AIC: AIC of best model

  s <- Sys.time()
  # available variables set
  available_vars <- setdiff(1:ncol(data), response)

  # generate subset randomly
  while(TRUE) {
    subset_vars <- .generate_subset(available_vars)
    k <- length(subset_vars)
    # check if logistic valid, if not keep sampling
    if (isValidLogisticRCDD(response = response, explanatory = subset_vars, data = data)) {
      # if valid, proceed
      break
    }
  }
}
```

```

}

# initial model
curr_model <- subset_vars
# initial best model
curr_best <- curr_model

curr_aic <- getLogisticAIC(response, curr_model, data)
best_aic <- curr_aic

# MC iterations
for (i in 1:n_iter) {

  # print(sprintf("...Running for iteration %d / %d", i, n_iter))

  # step 1 & 2: find valid neighbor variables
  valid_neighbors <- .find_valid_neighbor(
    response = response,
    explanatory = curr_model,
    data = data
  )

  # step 3: uniformly sample a model from valid neighbors
  if (length(valid_neighbors) == 0) {
    #FIXME: this edge case should be handled but no instruction mentioned
    #TODO: verify how to deal with this case
  }
  model_candidate <- valid_neighbors[[sample(1:length(valid_neighbors), 1)]] # it's a list

  # step 4: find neighbor of curr model candidate
  neighbors_of_candidate <- .find_valid_neighbor(
    response = response,
    explanatory = model_candidate,
    data = data
  )

  # step 5: calc. p for MA'
  candidate_aic <- getLogisticAIC(response, model_candidate, data)
  p_candidate <- -candidate_aic - log(length(neighbors_of_candidate))

  # step 6: calc. p for MAr
  p_curr <- -curr_aic - log(length(valid_neighbors))

  # step 7 & 8: current model picking w/ accept-rejecting sampling
  # NOTE: more readable to have accept cases organized
  accept <- FALSE

  if (p_candidate > p_curr) {
    accept <- TRUE
  } else {
    u <- runif(1)
    if (log(u) < p_candidate - p_curr) {
      accept <- TRUE
    }
  }
}

```

```

    }
  }

  if (accept) {
    # update current model if accepting new candidate
    curr_model <- model_candidate
    curr_aic <- candidate_aic
  }

  if (curr_aic < best_aic) {
    # update current best model if there's improvement wrt AIC
    curr_best <- curr_model
    best_aic <- curr_aic
  }

} # end of iteration loop
e <- Sys.time()
print(sprintf("time elapsed: %.4f s", e-s))
return(list(
  bestAIC = best_aic,
  bestAICvars = curr_best
))
}

```

Part B. AI Solution

Prompt

Please design a function named `M3search` that implement the MCMC composition algo. to perform model selection with AIC metric. The function should be design into this:

```

AI_M3search <- function(response, data, n_iter) {
  #'
  #' Args:
  #' ----
  #' - response (int): col index of binary response
  #' - data (data.frame | matrix): target dataset
  #' - n_iter (int): number of MC3 search iterations
  #'
  #' Returns:
  #' -----
  #' - list(best model, best AIC) (list)
  #'       - best model: set of index of explanatory vars
  #'       - best AIC: AIC of best model
}

```

Important requirements:

Use this exact function to check whether a logistic regression model is valid. Do not rewrite or replace it.

```

# isValidLogisticRCDD <- function(response, explanatory, data)
# {
#   if(0==length(explanatory))
#   {
#     #we assume that the empty logistic regresion is valid

```

```

# return(TRUE);
# }
# logisticreg = suppressWarnings(glm(data[,response] ~ as.matrix(data[,as.numeric(explanatory)]),family
# tanv = logisticreg$x;
# tanv[data[,response] == 1, ] <- (-tanv[data[,response] == 1, ]);
# vrep = cbind(0, 0, tanv);
# #with exact arithmetic; takes a long time
# #lout = linearity(d2q(vrep), rep = "V");
#
# lout = linearity(vrep, rep = "V");
# return(length(lout)==nrow(data));
# }

```

Please also design helper functions so that the main MC3search() function is not too complicated.

For the algo, the MC3 algorithm should follow these steps:

- Define the available explanatory variables as all column indices except response.
- Randomly generate a valid starting model.
- Set the current model and the best model equal to this starting model.

In each iteration: 1. Identify the neighbors of the current logistic regression $MA(r)$. These neighbors are obtained by adding one variable to the set of explanatory variables $A(r)$ or by deleting one variable from $A(r)$

2. Use the function `isValidLogisticRCDD` to eliminate from `nbd(MA(r))` all the logistic regressions whose MLEs do not exist. We denote by `nbdvalid(MA(r))` the resulting set of valid neighbors of $MA(r)$.
3. Uniformly sample a model MA' from `nbdvalid(MA(r))`. The functions `sample` and `sample.int` could be useful here.
4. Determine the set of valid neighbors of MA' , denoted by `nbdvalid(MA')`. Here you repeat Steps 1 and 2 above for MA' instead of $MA(r)$. If you developed nice functions for performing each step, you should not have to write any new code at here.
5. calculate $p_{rA} = -AIC(M_{rA}) - \log(\#(nbd^{valid}(M_{rA}))$. the latter represents the number of elements (models) in the set of models MA' .
6. calculate $p_{A(r)} = -AIC(M_{A(r)}) - \log(\#(nbd^{valid}(M_{A(r)}))$.
7. If $p_{rA} > p_{A(r)}$, MA' becomes the current model. That is, set $A(r+1) = A'$ and use equation (2) to find out the best logistic regression $MB(r+1)$ visited so far. More explicitly, if $AIC(MA') < AIC(MB(r))$ set $B(r+1) = A'$. Otherwise set $B(r+1) = B(r)$. Move to Step 9
8. If $p_{rA} \leq p_{A(r)}$, sample u from the uniform distribution on $(0, 1)$. The R function `runif` could be useful. If $\log(u) < p_{A'} - p_{A(r)}$, MA' becomes the current model. Perform the same updates as in Step 7. If $\log(u) \geq p_{A'} - p_{A(r)}$, $MA(r)$ remains the current model (i.e., the Markov chain does not move to the proposed state MA'). In this case you set $A(r+1) = A(r)$ and $B(r+1) = B(r)$
9. If the current maximum number of iterations has been reached (i.e., if $r = R$), STOP. Otherwise continue to the next iteration.

The MC3 algorithm should output the indices of the explanatory variables associated with the best logistic regression model identified as well as the value of the AIC of this best model. In other words, you should output $B(R)$ and $AIC(MB(R))$.

Also, remember to name all your functions with `AI_` prefix to distinguish from my own functions.

Code Generated

```

source("hw3-ai-lib.R") # source the helper functions

# main function design
AI_M3search <- function(response, data, n_iter) {
  # 1. Initialization
  all_vars <- setdiff(1:ncol(data), response)

  # Randomly generate a valid starting model
  # We'll try random subsets until one is valid
  valid_start <- FALSE
  while (!valid_start) {
    start_size <- sample(0:length(all_vars), 1)
    current_model <- sort(sample(all_vars, start_size))
    if (isValidLogisticRCDD(response, current_model, data)) {
      valid_start <- TRUE
    }
  }

  best_model <- current_model
  best_aic <- AI_get_AIC(response, best_model, data)

  # 2. Iteration Loop
  for (r in 1:n_iter) {
    # Step 1 & 2: Get valid neighbors of current model
    nbd_curr <- AI_get_valid_neighbors(current_model, all_vars, response, data)

    # Step 3: Uniformly sample a candidate model M_prime
    idx_prime <- sample(seq_along(nbd_curr), 1)
    m_prime <- nbd_curr[[idx_prime]]

    # Step 4: Get valid neighbors of the candidate model
    nbd_prime <- AI_get_valid_neighbors(m_prime, all_vars, response, data)

    # Step 5 & 6: Calculate acceptance probabilities
    #  $p = -AIC - \log(\#nbd)$ 
    aic_prime <- AI_get_AIC(response, m_prime, data)
    aic_curr <- AI_get_AIC(response, current_model, data)

    p_prime <- -aic_prime - log(length(nbd_prime))
    p_curr <- -aic_curr - log(length(nbd_curr))

    # Step 7 & 8: Metropolis-Hastings Acceptance Step
    accepted <- FALSE
    if (p_prime > p_curr) {
      accepted <- TRUE
    } else {
      u <- runif(1)
      if (log(u) < (p_prime - p_curr)) {
        accepted <- TRUE
      }
    }
  }

  if (accepted) {

```

```

    current_model <- m_prime
    # Update global best if the new current model is better than any seen before
    if (aic_prime < best_aic) {
        best_model <- m_prime
        best_aic <- aic_prime
    }
}
# If not accepted, current_model and best_model remain unchanged
}

# 3. Return results
return(list(
    "best model" = best_model,
    "best AIC" = best_aic
))
}

```

Part C. Comparison

```
df <- fread("hw2/534binarydatasmall.txt") |> as.data.frame()
```

```
# my solution
```

```
res <- MC3search(ncol(df), df, n_iter = 10)
```

```
## [1] "time elapsed: 5.5131 s"
```

```
AI_res <- AI_M3search(ncol(df), df, n_iter = 10)
```

```
res
```

```
## $bestAIC
```

```
## [1] 117.3657
```

```
##
```

```
## $bestAICvars
```

```
## [1] 5 2 10 7 3 8 1 6
```

```
AI_res
```

```
## $`best model`
```

```
## [1] 2 3 4 5 6 8 9 10
```

```
##
```

```
## $`best AIC`
```

```
## [1] 148.6865
```

Correctness

- Both codes implement the correct logics, and we get the similar AIC results on both testing datasets.
- AI' solution does not cope with the case when valid neighbors is empty, though it is barely possible, but my code mentions this edge case and flags it as TODO since current instruction did not mention how to do with this case.

Readability

- Both my and AI's solution simplify the accepted cases logic, and use accept flag to decide whether to update the model or best model information, which I think is the main improvement from bland implementation following the problem set instructions. This makes both code similarly readable in my opinion.

- Other things like doc-strings and comments, I find them also similar, though AI's could sometimes be more verbose on things not so important.

Algo & DS

- Both implementations represent a model as a vector of variable indices and store neighboring models in a list, which I think is simple and appropriate for this problem.
- The way the problem set written will store best model and best AIC as vectors, but in both solutions, both me and AI simplify this redundancy and only save the necessary variables to save memory space.
- Main algo and container structures do not have major difference between 2 solutions.

Problem 2

Part A: Solution

```
# setups
rm(list = ls()); gc()

##          used (Mb) gc trigger (Mb) limit (Mb) max used (Mb)
## Ncells 1082397 57.9   2194358 117.2      NA   2194358 117.2
## Vcells 1932609 14.8    8388608  64.0     16384  8388461  64.0

source("hw3.R")

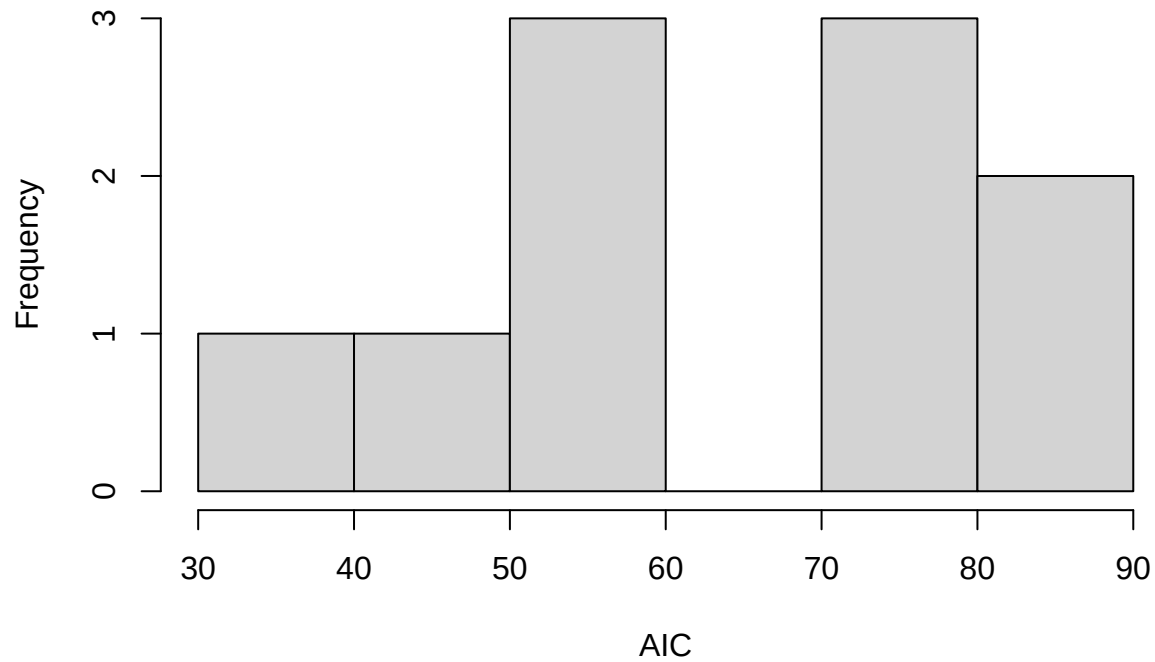
# data
df <- fread("hw2/534binarydata.txt") |> as.data.frame()

# TODO: cache results
if (!fs::file_exists("hw3-cached.rds")) {
  set.seed(9527)
  # NOTE: this part is super time-consuming, cache the result to avoid long compiling
  n_reps <- 10
  res_list <- vector(mode = "list", length = n_reps)
  for (i in 1:n_reps) {
    res_ <- MC3search(ncol(df), df, n_iter = 25)
    res_list[[i]] <- res_
  }
  # cache the result
  readr::write_rds("hw3-cached.rds")
}

res <- read_rds("hw3-cached.rds")

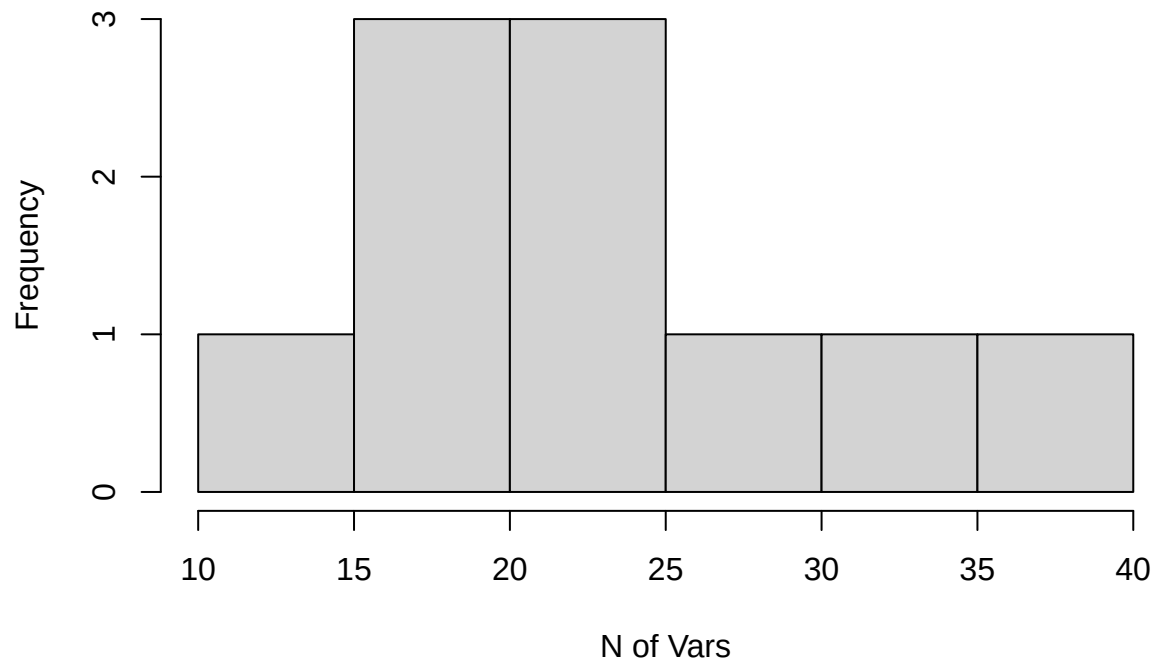
aics <- sapply(res, `[`, "bestAIC")
hist(aics, main = "Histogram of best AICs", xlab = "AIC")
```

Histogram of best AICs



```
n_vars <- sapply(lapply(res, `[`, "bestAICvars"), length)
hist(n_vars, main = "Histogram of n. of vars in best model", xlab = "N of Vars")
```

Histogram of n. of vars in best model



Temp Conclusions and Observations

The results show substantial variability across the 10 independent MC3 runs, indicating that the algorithm has not converged within 25 iterations. Also, the best model seems to be quite different, even the num of vars in best model has wide range from above histogram.

Part B: AI' solution

Prompt

Run 10 times of AI_MC3search, load in data named "534binarydata.txt", then keep all the results in a list. save the results into "hw3-ai-cached.rds" file for later use.

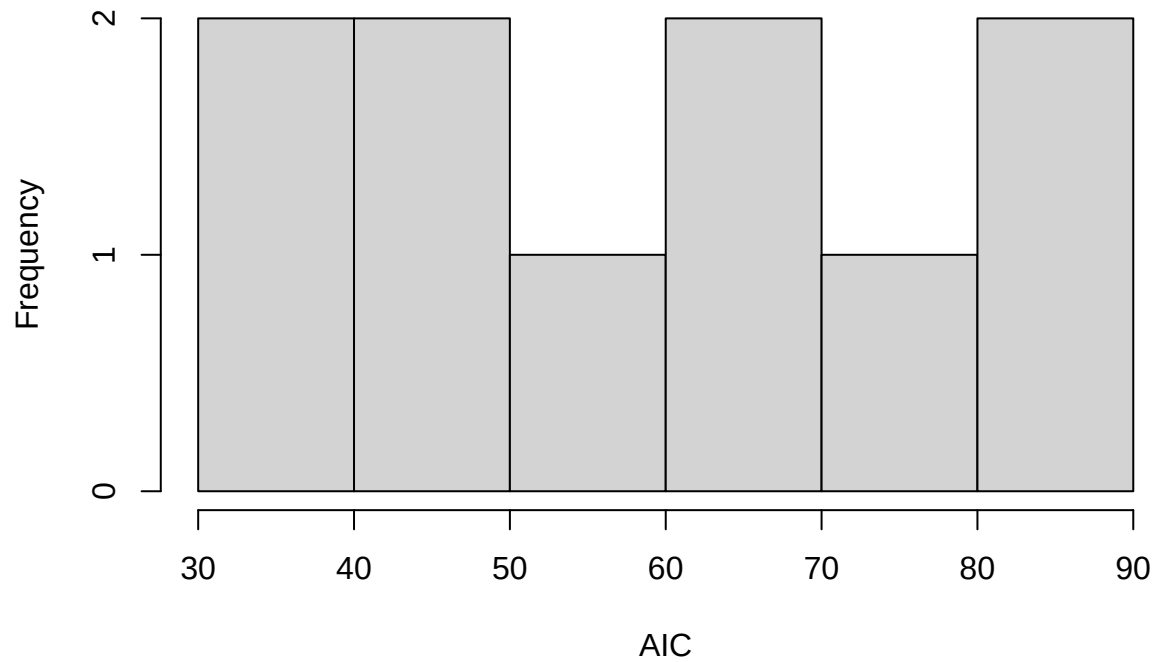
Code Generated

```
if (!fs::file_exists("hw3-ai-cached.rds")) {  
  # Load data  
  # Note: Ensure the file path is correct for your local machine  
  data_df <- read.table("534binarydata.txt", header = TRUE)  
  
  # Assume response is the first column (update if necessary)  
  resp_idx <- 1  
  
  # Run the search 10 times  
  results_list <- list()  
  set.seed(123) # For reproducibility  
  for (i in 1:10) {  
    cat("Running iteration:", i, "\n")  
    results_list[[i]] <- AI_MC3search(response = resp_idx, data = data_df, n_iter = 500)  
  }  
  
  # Save results to the cached file  
  readr::write_rds(results_list, "hw3-ai-cached.rds")  
  cat("Results saved to hw3-ai-cached.rds\n")  
}
```

Part C: Comparison

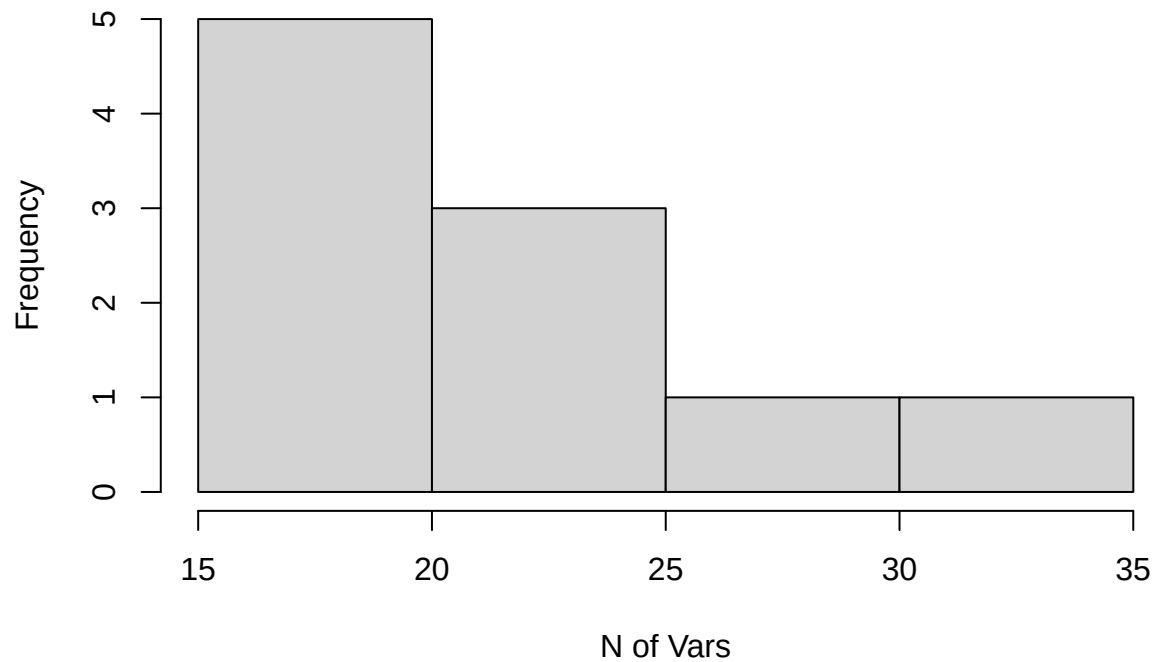
```
ai_res <- read_rds("hw3-ai-cached.rds")  
ai_aics <- sapply(ai_res, `[`, "bestAIC")  
hist(ai_aics, main = "Histogram of AI best AICs", xlab = "AIC")
```

Histogram of AI best AICs



```
n_vars <- sapply(lapply(ai_res, `[`, "bestAICvars"), length)
hist(n_vars, main = "Histogram of n. of vars in AI best model", xlab = "N of Vars")
```

Histogram of n. of vars in AI best model



Correctness

- Both solutions produce similar results, the AIC ranging from 30 to 90, and both are quite unstable with 25 iterations. The outputs are similar in conclusion.
- Main logic is straightforward and noting has major differences between solutions

Readability

- The code is super easy (to run 10 times search), the readability between solutions are similar.

Algo and DS

- Both solutions choose to use list to store the results
- AI's solution does not specify the list length at first, and dynamically append elements onto the list. This could be potentially slow if the length of list get larger; yet in 10-element list case, it only differs subtly. My solution, on the other hand, specify the length at creation, making the container creation more efficient.